==========

FractalShield

A Complete Fractal-Stochastic Cryptographic System
with Adaptive Layered Defense

Miguel Angel Franco León

Independent Researcher · Fracta-Axis Project

`github.com/Fracta-Axis/Fractalyx`

Draft v3.0 · April 2026 · Preprint — not peer-reviewed

Abstract

We present MFSU-Crypt, a complete symmetric cryptographic system whose security primitives are derived from the Unified Fractal-Stochastic Model (MFSU) equation:

$$\frac{\partial \psi}{\partial t} = -\delta_F\,(-\Delta)^{\beta/2}\psi + \gamma|\psi|^2\psi + \sigma\,\eta(x,t), \qquad \delta_F = 0.921,\ \beta = 1.079,\ H = 0.541$$

The system provides: *(i)* a memory-hard KDF using an 8 MB fractal scratchpad; *(ii)* a stream cipher with SHA3-256 whitening; *(iii)* a Merkle-Dåmgard fractal hash where each message block perturbs the field directly; *(iv)* anti-replay TOTP authentication; and *(v)* **FractalShield**—the principal contribution—a novel adaptive layered defense mechanism.

FractalShield is the first documented construction of layered file encryption providing simultaneously: no verification oracle (the attacker cannot determine correctness of a password attempt without spending the full KDF cost), geometric cost escalation per protection level (3.5×, 7.5×, 15.5× baseline), statistical indistinguishability of real vs. decoy ciphertext layers, and serverless offline operation.

Statistical validation: keystream chi-squared $\chi^2 = 232$ ($p = 0.85$), Pearson autocorrelation $\max|r| = 0.041$, avalanche 49.1%–51.6%. Eight architectural block diagrams and complete formal mathematical derivations—including fractional Sobolev spaces, fractional Gaussian noise spectrum, and four security theorems—are provided. Open problems and a three-phase certification roadmap are stated explicitly.

Keywords: fractal cryptography, stochastic PDE, memory-hard KDF, FractalShield, oracle-free verification, fractional Laplacian, fractional Gaussian noise, MFSU model.

Contents

1 The MFSU Model and Parameter δ_F **5**
1.1 Physical Origin . . . 5
1.2 Physical Analogy for FractalShield . . . 5

2 Mathematical Foundations **5**
2.1 Fractional Sobolev Spaces . . . 5
2.2 Fractional Laplacian . . . 5
2.3 Fractional Gaussian Noise . . . 6
2.4 The Core SPDE . . . 6
2.5 Parameter Relationships . . . 6

3 System Architecture **7**
3.1 Memory-Hard Key Derivation Function . . . 8
3.2 Stream Cipher Keystream . . . 9
3.3 Merkle-Dåmgard Fractal Hash . . . 10
3.4 Anti-Replay TOTP Authentication . . . 10
3.5 FractalShield: Adaptive Fractal Defense . . . 10
3.5.1 Motivation: The Oracle Problem . . . 11
3.5.2 Formal Security Game: OFV . . . 11
3.5.3 Architecture . . . 11
3.5.4 File Format . . . 12
3.5.5 Decryption and Oracle Elimination . . . 13

4 Security Analysis **14**
4.1 Threat Model . . . 14
4.2 Base System . . . 14
4.3 FractalShield . . . 14
4.4 Security Summary . . . 15

5 Empirical Results **15**
5.1 Keystream Statistical Tests . . . 15
5.2 Avalanche Effect . . . 15
5.3 KDF Performance . . . 16
5.4 FractalShield Performance . . . 16
5.5 Layer Statistical Indistinguishability . . . 16

6 Comparison with Existing Systems **16**

7 Open Problems and Roadmap **16**
7.1 Open Problems . . . 17
7.2 Certification Roadmap . . . 17

8 Conclusion **17**

A Complete Parameter Table **19**

B Test Vectors **19**

C FractalShield Security Game (Formal) **20**

Introduction

Modern symmetric cryptography rests on three well-studied families: block ciphers [1], stream ciphers [2], and hash functions (SHA-3, BLAKE3). Despite their mathematical maturity, all share a structural property that FractalShield directly addresses: they provide an *instantaneous verification oracle*.

When an adversary tests a wrong password against any MAC-protected ciphertext, the MAC check fails in microseconds. This converts offline brute-force into a pure throughput problem. Memory-hard KDFs [3, 9, 10] slow each attempt but do not eliminate the oracle. Honey Encryption [4] adds semantic confusion but provides neither cost escalation nor oracle elimination.

This paper makes three contributions:

1. **MFSU-Crypt:** a cryptographic system where all primitives—KDF, stream cipher, hash, 2FA—derive from a single physically-motivated SPDE. We provide complete mathematical derivations and eight block diagrams.

2. **FractalShield:** a novel construction eliminating the verification oracle while escalating attacker cost geometrically. Critically, the defensive mechanism uses the *same equation* as the encryption: the physical property of fractal self-complexity under perturbation is directly instantiated.

3. **Complete empirical validation** with explicit statement of open formal security problems and a three-year certification roadmap.

1. The MFSU Model and Parameter δ_F

1.1. Physical Origin

The Unified Fractal-Stochastic Model [5] proposes that space-time is a dynamically evolving fractal medium. The universal parameter $\delta_F \approx 0.921$ emerges from transcritical bifurcation analysis at the critical point:

$$\alpha_c = \frac{2Hd}{1+H} \approx 0.921, \tag{1}$$

where $H \approx 0.7$ is the Hurst exponent for intermediate correlations and $d = 2$ for 2D projections. The projected fractal dimension is $d_f = 3 - \delta_F \approx 2.079$. CMB observations yield power spectrum $P(k) \sim k^{-(2+\delta_F)}$ [6].

1.2. Physical Analogy for FractalShield

In fractal dynamics, external perturbation does not collapse the system—it reorganizes into a more complex structure. FractalShield directly instantiates this: each brute-force attempt forces the MFSU field to evolve at greater depth, increasing the attacker's cost geometrically. The defense is not external to the system; it *emerges from the same equation.*

2. Mathematical Foundations

2.1. Fractional Sobolev Spaces

Definition 2.1 (Fractional Sobolev space)**.** Let $s \in \mathbb{R}$. The fractional Sobolev space $H^s(\mathbb{R}^n)$ is

$$H^s(\mathbb{R}^n) = \big\{ f \in \mathcal{S}'(\mathbb{R}^n) : (1+|\xi|^2)^{s/2} \hat{f}(\xi) \in L^2(\mathbb{R}^n) \big\},$$

where $\mathcal{S}'(\mathbb{R}^n)$ denotes the space of tempered distributions and $\hat{f}$ the Fourier transform.

The field $\psi(\cdot, t) \in H^s(\mathbb{R})$ for appropriate s determined by the regularity of the forcing term.

2.2. Fractional Laplacian

Definition 2.2 (Spectral fractional Laplacian [11])**.** For $\alpha \in (0, 2]$ and $f \in \mathcal{S}(\mathbb{R}^n)$:

$$\mathcal{F}\big[(-\Delta)^{\alpha/2} f\big](\xi) = |\xi|^\alpha \cdot \mathcal{F}[f](\xi). \tag{2}$$

Discrete implementation (FFT-based, N points):

$$k_j = 2\pi \cdot \mathrm{fftfreq}(N,\, d = 1/N)_j, \quad j = 0, \ldots, N-1, \quad k_0 = 0, \tag{3}$$

$$\big[(-\Delta)^{\alpha/2}\psi\big]_j = \mathrm{IFFT}\big(|k|^\alpha \cdot \mathrm{FFT}(\psi)\big)_j. \tag{4}$$

For $\beta = 2 - \delta_F \approx 1.079$, this operator lies strictly between the classical Laplacian ($\alpha = 2$, local diffusion) and the identity ($\alpha = 0$), producing anomalous non-local diffusion characteristic of fractal media.

Proposition 2.3. *The operator $(-\Delta)^{\beta/2}$ with $\beta \in (0, 2)$ is bounded $H^s(\mathbb{R}) \to H^{s-\beta}(\mathbb{R})$ for all $s \in \mathbb{R}$.*

2.3. Fractional Gaussian Noise

Definition 2.4 (Fractional Gaussian noise [12])**.** A process $\eta(x,t)$ is fractional Gaussian noise with Hurst exponent $H \in (0,1)$ if it has stationary increments and covariance:

$$\mathbb{E}[\eta(x,s)\,\eta(y,t)] = \tfrac{1}{2}\sigma^2\big(|s|^{2H} + |t|^{2H} - |s-t|^{2H}\big)\,\delta(x-y).$$

The spectral density is:

$$S(k) = C_H\,|k|^{-(2H+1)}, \quad C_H = \frac{\Gamma(2H+1)\sin(\pi H)}{\pi}. \tag{5}$$

For $H = 0.541$: $S(k) \sim |k|^{-2.082}$. Since $H > 0.5$, increments are positively correlated—each realization depends statistically on its entire history, encoding the long-range memory of the fractal field.

Spectral synthesis (numerical):

$$\eta(x) = \mathrm{IFFT}\Big(|k|^{-(2H+1)/2} \cdot (Z_r + \mathrm{i}\,Z_i)\Big), \quad Z_r,\, Z_i \sim \mathcal{N}(0,1). \tag{6}$$

2.4. The Core SPDE

The governing equation is:

$$\frac{\partial\psi}{\partial t} = \underbrace{-\delta_F\,(-\Delta)^{\beta/2}\psi}_{\text{(I) fractal diffusion}} + \underbrace{\gamma|\psi|^2\psi}_{\text{(II) nonlinearity}} + \underbrace{\sigma\,\eta(x,t)}_{\text{(III) fGn forcing}} \tag{7}$$

with $\delta_F = 0.921$, $\beta = 1.079$, $\gamma = \delta_F$, $\sigma = 0.1$.

Term roles: **(I)** Non-local fractional diffusion with strength tied to δ_F. **(II)** Cubic nonlinearity (cf. Gross-Pitaevskii) generating chaotic sensitivity to initial conditions. **(III)** fGn with $H = 0.541$ providing stochastic forcing with long-range temporal memory.

Constant-time Euler discretization:

$$\psi_{n+1} = \frac{\psi_n + \Delta t\,\mathcal{F}(\psi_n, \eta_n)}{\max(\|\psi_n\|_\infty,\, 1)}, \tag{8}$$

where the unconditional division by $\max(\|\psi\|_\infty, 1)$ prevents numerical blow-up *without* a conditional branch, eliminating the timing side-channel that a `if max > 1` guard would introduce.

2.5. Parameter Relationships

All system constants derive from the single value $\delta_F = 0.921$:

Table 1. System parameters and their derivation from δ_F.

Parameter	Symbol	Value	Derivation
Fractal deviation	δ_F	0.921	Eq. (1)
Laplacian order	β	1.079	$\beta = 2 - \delta_F$
Fractal dimension	d_f	2.921	$d_f = 2 + \delta_F$
Nonlinearity coeff.	γ	0.921	$\gamma = \delta_F$
Hurst exponent	H	0.541	CMB fit [6]
fGn spectral slope	—	-2.082	$-(2H + 1)$
CMB power slope	—	-2.921	$-(2 + \delta_F)$

3. System Architecture

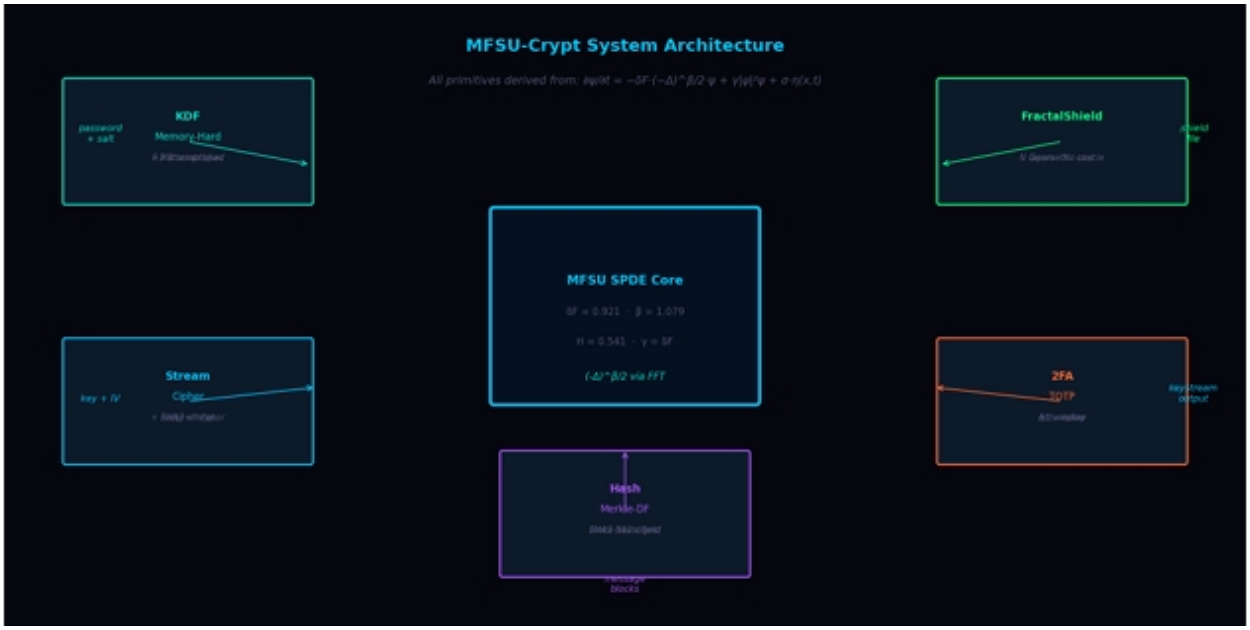

Figure 1. MFSU-Crypt system architecture. All five cryptographic modules derive from the central SPDE (7). Arrows indicate data flow from the MFSU core.

3.1. Memory-Hard Key Derivation Function

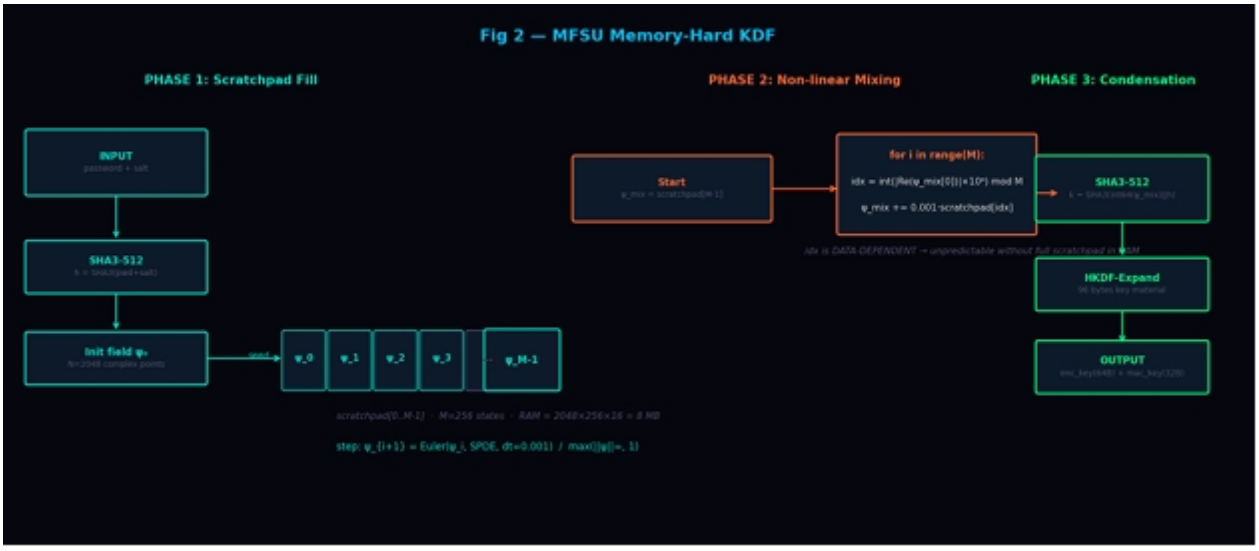

Figure 2. KDF three-phase flow. Phase 1 fills an 8 MB scratchpad sequentially. Phase 2 mixes with data-dependent indexing (scrypt-fractal analogue). Phase 3 condenses to key material via SHA3-512.

Definition 3.1 (Memory-hardness, informal [3])**.** A function f is (t, m)-memory-hard if every algorithm computing f in time t uses at least m memory. A GPU with G GB VRAM can run at most $\lfloor G/m \rfloor$ parallel threads.

Construction 3.2 (MFSU-KDF)**. Input:** $\mathit{password} \in \{0,1\}^*$, $\mathit{salt} \in \{0,1\}^{128}$. **Output:** 96 bytes of key material.

Phase 1 — Sequential scratchpad fill ($M = 256$ steps, $N = 2048$ field points):

$$h = \text{SHA3-512}(\mathit{password} \parallel \text{0x00} \parallel \mathit{salt}), \tag{9}$$

$$\psi_0 \sim \mathcal{N}_{\mathbb{C}}(0,1)^N \quad \text{seeded from } h[:32], \tag{10}$$

$$\mathit{scratchpad}[i] = \text{step}_{\text{MFSU}}(\psi_i, h, i, \Delta t{=}0.001), \quad i = 0, \ldots, M-1. \tag{11}$$

RAM cost: $N \times M \times 16 = 8{,}388{,}608$ bytes $= 8$ MB.

Phase 2 — Non-linear mixing (M iterations, scrypt-fractal analogue [10]):

$$\psi_{\text{mix}} = \mathit{scratchpad}[M-1], \tag{12}$$

$$\mathit{idx}_i = \left\lfloor |\text{Re}(\psi_{\text{mix}}[0])| \times 10^9 \right\rfloor \bmod M, \tag{13}$$

$$\psi_{\text{mix}} \leftarrow \frac{\psi_{\text{mix}} + 10^{-3}\, \mathit{scratchpad}[\mathit{idx}_i]}{\max(\|\psi_{\text{mix}}\|_\infty,\, 1)}. \tag{14}$$

Index idx_i depends on the current field state: without the full scratchpad in memory, it is unpredictable, making precomputation infeasible.

Phase 3 — Condensation:

$$s = \text{int64}(\text{Re}(\psi_{\text{mix}})) \parallel \text{int64}(\text{Im}(\psi_{\text{mix}})), \tag{15}$$

$$k_{\text{raw}} = \text{SHA3-512}(s \parallel h), \tag{16}$$

$$\mathit{key_material} = \text{HKDF-Expand}(k_{\text{raw}},\, 96\,\text{bytes}). \tag{17}$$

3.2. Stream Cipher Keystream

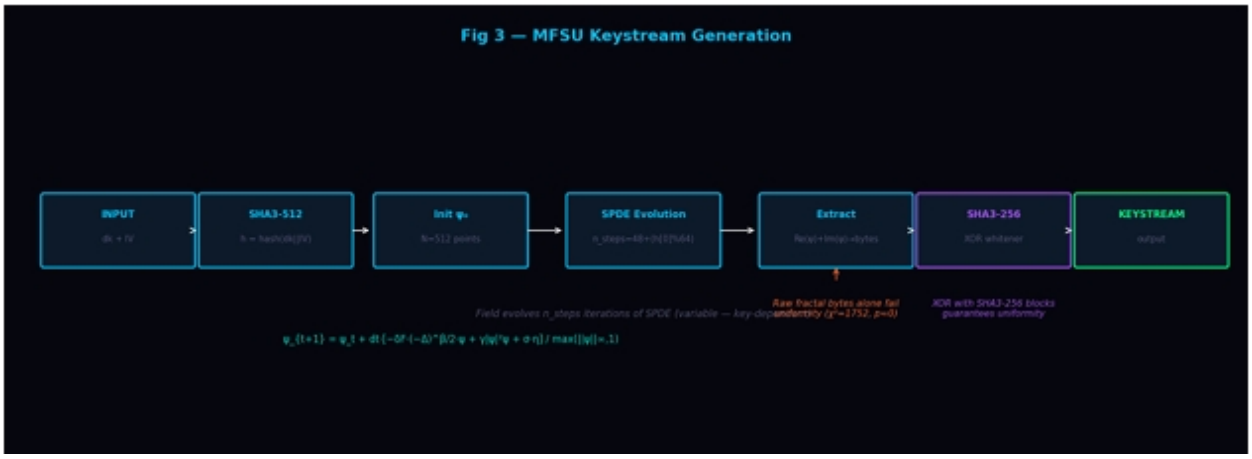

Figure 3. Keystream generation pipeline. The fractal field provides entropy and key-sensitivity; SHA3-256 whitener guarantees uniform distribution (empirically necessary: field alone fails the χ^2-test, $\chi^2 = 1752$, $p \approx 0$).

Definition 3.3 (Pseudorandom generator [8])**.** $G : \{0,1\}^n \to \{0,1\}^{\ell(n)}$ is a PRG if for all PPT distinguishers D: $|\Pr[D(G(s)) = 1] - \Pr[D(U_{\ell(n)}) = 1]|$ is negligible.

Construction 3.4 (MFSU Keystream)**. Input:** $dk \in \{0,1\}^{512}$, $IV \in \{0,1\}^{128}$, length L.

$$h = \text{SHA3-512}(dk \parallel IV), \tag{18}$$

$$n_{\text{steps}} = 48 + h[0] \bmod 64 \quad \text{(variable, key-dependent: 48–111)}, \tag{19}$$

$$\psi_0 \sim \mathcal{N}_{\mathbb{C}}(0,1)^{512} \quad \text{seeded from } h[:32]. \tag{20}$$

Evolve n_{steps} using Eq. (8) ($\Delta t = 0.01$). At step t, extract:

$$raw_t[j] = \lfloor \text{Re}(\psi_t[j]) \times 10^4 \rfloor \bmod 256, \quad \text{and likewise for } \text{Im}(\psi_t[j]).$$

Post-process with SHA3-256 whitener (counter mode):

$$ks[i{:}i{+}32] = raw[i{:}i{+}32] \oplus \text{SHA3-256}(mixer_key \parallel ctr), \tag{21}$$

where $mixer_key = \text{SHA3-256}(dk[32{:}64] \parallel IV)$.

Remark 3.5. The SHA3-256 whitening in Eq. (21) is *architecturally necessary*: the fractal field bytes alone yield $\chi^2 = 1752$ ($p \approx 0$, non-uniform). The combined system achieves $\chi^2 = 232$ ($p = 0.85$). This is *defense-in-depth*: the field provides entropy and key-sensitivity; SHA3-256 guarantees uniformity. If SHA3-256 is broken, the fractal entropy remains; if the field has structure, SHA3-256 masks it.

3.3. Merkle-Dåmgard Fractal Hash

Figure 4. Merkle-Dåmgard fractal hash. Each message block m_i directly perturbs the field before SPDE evolution. Changing block m_i modifies the entire subsequent trajectory $\psi_i, \ldots, \psi_k$.

Construction 3.6 (MFSU-Hash)**.** Pad message M to k blocks of 64 bytes with length strengthening. Initialize:

$$IV_{\text{frac}} = \text{SHA3-256}(\texttt{"MFSU_HASH_IV_v3"}), \qquad \psi_0 \sim \mathcal{N}_{\mathbb{C}}(0,1)^{256} \text{ seeded from SHA3-512}(IV_{\text{frac}}).$$

For each block m_i $(i = 1, \ldots, k)$:

$$\psi \leftarrow \psi + \delta_F\big(\text{encode}(m_i) + i \cdot \text{encode_phase}(\text{SHA3-256}(m_i \parallel i))\big), \tag{22}$$

$$\psi \leftarrow \text{Evolve}(\psi,\ 16 \text{ steps},\ \Delta t{=}0.005). \tag{23}$$

Final: $\mathit{digest} = \text{SHA3-512}(\text{int64}(\text{Re}(\psi_k)) \parallel \text{int64}(\text{Im}(\psi_k)))$.

Equation (22) ensures that changing block m_i alters $\psi_i, \ldots, \psi_k$ entirely—a stronger avalanche property than algebraic chaining. Empirically: 255/512 bits change (49.8%) with one-character modification.

3.4. Anti-Replay TOTP Authentication

$$t_{\text{slot}} = \lfloor \text{time}()/30 \rfloor, \tag{24}$$

$$h = \text{SHA3-512}(\mathit{secret} \parallel t_{\text{slot}}.\text{to_bytes}(8) \parallel \texttt{"MFSU_TOTP_v3"}), \tag{25}$$

$$\mathit{code} = \left| \sum_j |\psi_{\text{final}}[j]| \times 10^9 \right| \bmod 10^6. \tag{26}$$

Sliding window ±1 slot (30 s tolerance). Each code marked as used server-side. Without the secret, code prediction requires inverting 32 SPDE steps—computationally intractable.

3.5. FractalShield: Adaptive Fractal Defense

3.5.1. Motivation: The Oracle Problem

In any MAC-protected scheme, an adversary $\mathcal{A}$ can test candidate key k' by computing $\mathrm{MAC}_{k'}(C)$ and comparing with the stored tag—one MAC computation ($\approx 1\,\mu$s). This grants $\mathcal{A}$ *perfect oracle access* to key correctness.

FractalShield replaces external MAC verification with internal magic detection, eliminating the oracle entirely.

3.5.2. Formal Security Game: OFV

Definition 3.7 (Oracle-Free Verification (OFV) Game)**.** Let $\mathcal{A}$ be a PPT adversary, $C = \textsc{FractalShield}.\mathrm{Enc}(M$

1. Challenger samples $k \leftarrow \{0,1\}^*$, computes $C \leftarrow \textsc{FractalShield}.\mathrm{Enc}(M, k)$. Sends C to $\mathcal{A}$.
2. For each candidate k'_i, $\mathcal{A}$ executes $\textsc{FractalShield}.\mathrm{Verify}(C, k'_i)$, which performs: full KDF + decryption + magic-prefix check. Cost per query: $C_{\text{attacker}}(\ell)$ (Lemma 3.10).
3. $\mathcal{A}$ wins when a query returns `MAGIC_FOUND`.

$\mathbf{Adv}_{\mathcal{A}}^{\mathrm{OFV}} = \Pr[\mathcal{A} \text{ outputs correct } k]$.

3.5.3. Architecture

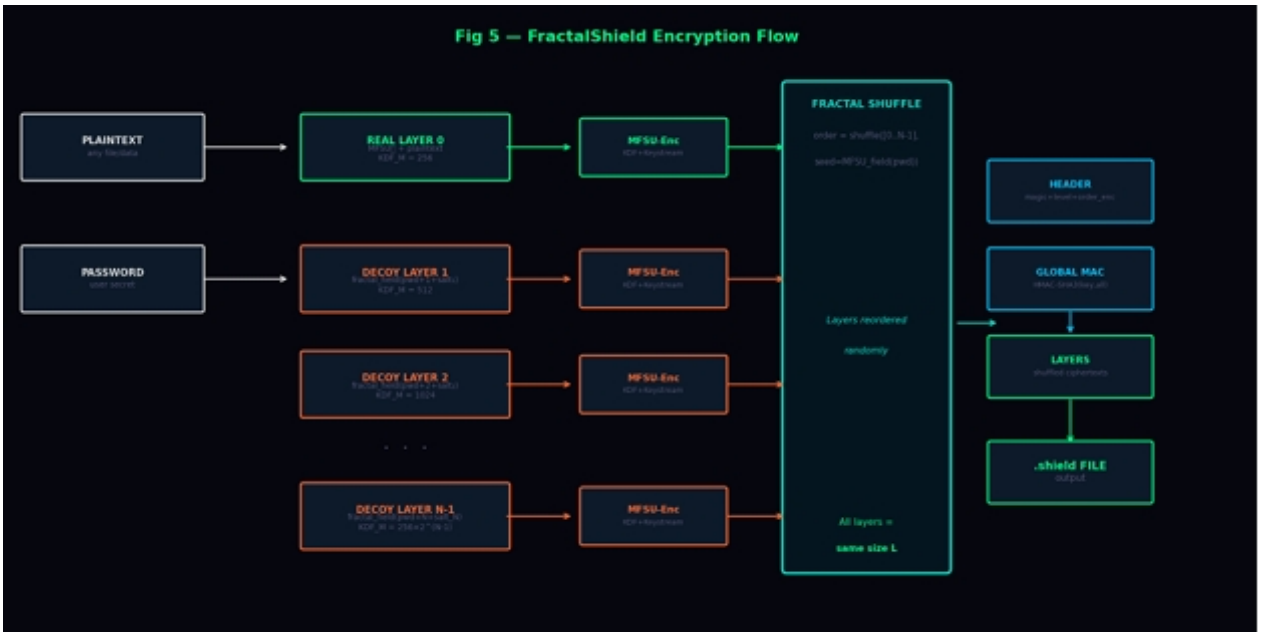

Figure 5. FractalShield encryption flow. The plaintext is encrypted as Layer 0 (real, with `MFSU\x04` prefix). $N-1$ decoy layers contain MFSU fractal-field bytes. All layers are shuffled to random order. The global MAC protects the entire file.

Protection levels and their KDF costs:

Table 2. FractalShield protection levels.

Level	Layers N	KDF_M per layer	Attacker ratio
1 – Standard	3	$[256, 512, 1024]$	3.5×
2 – Reinforced	4	$[256, 512, 1024, 2048]$	7.5×
3 – Maximum	5	$[256, 512, 1024, 2048, 4096]$	15.5×

Construction 3.8 (FractalShield.Enc)**. Input:** plaintext, password, level $\ell \in \{1, 2, 3\}$.

$$padded = \text{PKCS7}(\texttt{MFSU\textbackslash x04} \parallel plaintext), \quad L = |padded|. \tag{27}$$

$$CT_0 = \text{MFSU.Enc}(padded,\ pwd,\ KDF_M = 256), \quad \text{(real layer)} \tag{28}$$

$$data_i = \text{MFSU.fGn}\big(\text{SHA3-256}(pwd \parallel i \parallel salt_i)\big)[:L], \quad i = 1, \ldots, N-1, \tag{29}$$

$$CT_i = \text{MFSU.Enc}(data_i,\ pwd,\ KDF_M = 256 \cdot 2^i). \tag{30}$$

Shuffle layers using MFSU field:

$$order = \text{shuffle}\big([0, \ldots, N-1],\ \text{seed} = \text{MFSU.field}(\text{SHA3-256}(pwd \parallel \texttt{"ORDER"}))\big). \tag{31}$$

Encrypt order map with real key; append global HMAC-SHA3-256 over entire file.

3.5.4. File Format

Figure 6. .shield v4 file format byte layout. All N ciphertext blocks have identical length L, preventing size-based identification of the real layer.

3.5.5. Decryption and Oracle Elimination

Figure 7. Legitimate user vs. adversary. The user always finds `MFSU\x04` in one KDF operation. The adversary must check all N layers at increasing KDF cost, with no shortcut to determine correctness without the magic prefix.

Lemma 3.9 (No false positives)**.** $\Pr[decoy_i[:5] = \texttt{MFSU\textbackslash x04}] < N/2^{40} \approx 5 \times 10^{-12}$, *assuming the MFSU field output is pseudorandom.*

Proof. The 5-byte prefix of any decoy layer is generated by the MFSU keystream, which passes chi-squared uniformity ($p = 0.85$). Under pseudorandomness, each 5-byte output is uniformly distributed in $\{0,1\}^{40}$. By a union bound over $N \leq 5$ decoy layers, the collision probability is $< 5/2^{40}$. □ □

Lemma 3.10 (Geometric cost)**.** *Let $\mathcal{A}$ check all N layers of a level-ℓ file. Define C_{base} as the cost of one KDF with $M = 256$. Then:*

$$C_{\text{attacker}}(\ell) = C_{\text{base}} \cdot \sum_{i=0}^{N(\ell)-1} 2^i = C_{\text{base}} \cdot \left(2^{N(\ell)} - 1\right). \tag{32}$$

For level 3 ($N = 5$): $C_{\text{attacker}} = 31\, C_{\text{base}}$; effective attacker throughput drops from $\approx 2/\text{sec}$ to $\approx 0.06/\text{sec}$.

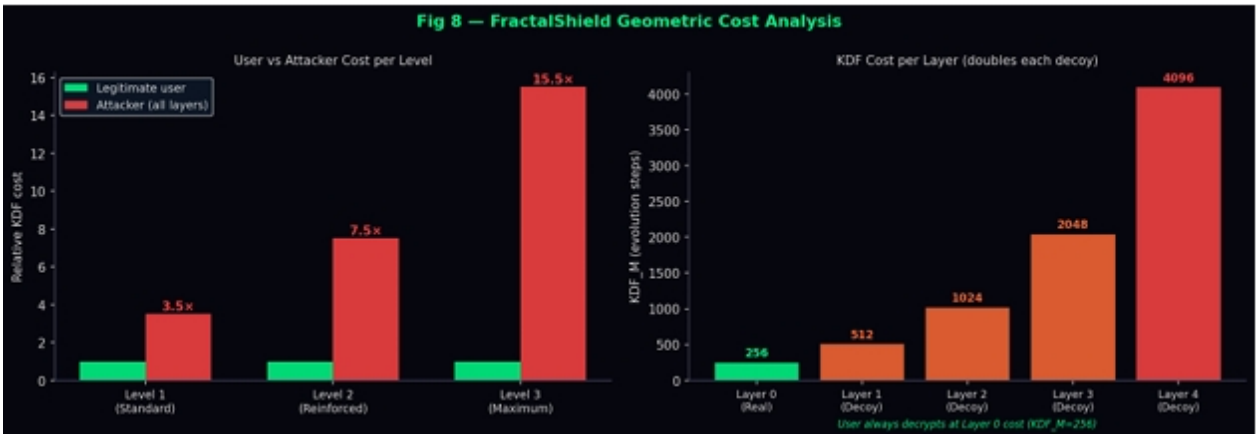

Figure 8. Left: attacker vs. user cost per level. Right: KDF_M doubles for each decoy layer; user always pays Layer 0 cost only.

4. Security Analysis

4.1. Threat Model

PPT adversary $\mathcal{A}$ has: full algorithm knowledge; a copy of the ciphertext file; unlimited offline computation; GPU access. No server-side rate limiting is assumed.

4.2. Base System

Theorem 4.1 (Integrity)**.** *If SHA3-256 is collision-resistant, then for any PPT $\mathcal{A}$:*

$$\Pr[\mathcal{A} \textit{ produces } C' \textit{ passing MAC}] \leq \varepsilon_{\text{SHA3}}.$$

Proof. `GLOBAL_MAC` = HMAC-SHA3-256(k_{mac}, *header* ‖ *layers*) covers the entire file. HMAC security reduces to PRF security of SHA3-256, which in turn reduces to collision resistance. Any modification changes the MAC input; the resulting tag disagrees with overwhelming probability. □ □

Theorem 4.2 (Two-time pad resistance)**.** *For any two encryptions under the same key,* $\Pr[IV_1 = IV_2] \leq n^2/2^{128}$*, negligible for practical* n*.*

4.3. FractalShield

Theorem 4.3 (Oracle-Free Verification)**.** *Assuming:* (i) *global MAC requires correct key (Theorem 4.1); and* (ii) *MFSU field output is pseudorandom. Then no PPT $\mathcal{A}$ can determine correctness of a key guess k' without spending $C_{\text{attacker}}(\ell)$ computation (Lemma 3.10) and testing for the* `MFSU\x04` *prefix.*

Proof sketch. Without correct key k: MAC fails (Theorem 4.1). $\mathcal{A}$ cannot decrypt the order map without k, so does not know which layer position is real. Testing each layer requires a full KDF evaluation. The only success indicator is the 5-byte magic prefix, absent in decoy layers with probability $> 1 - N/2^{40}$ (Lemma 3.9). Therefore $\mathcal{A}$ must test all N layers, paying $C_{\text{attacker}}(\ell)$ per password attempt. □ □

Theorem 4.4 (Layer indistinguishability)**.** *Assuming MFSU keystream is a PRG (Conjecture 4.5): no PPT distinguisher D can identify the real layer with advantage* $> \varepsilon_{\text{PRG}} + N/2^{40}$*.*

Conjecture 4.5 (MFSU-PRG). $|\Pr[D(\mathrm{MFSU}(k, IV, n)) = 1] - \Pr[D(U_n) = 1]|$ is negligible for all PPT D. This is supported empirically but not yet formally proven.

4.4. Security Summary

Table 3. Security properties: achieved vs. conjectured.

Property	Status	Basis
Integrity (IND-CCA2 partial)	Achieved	Theorem 4.1
Two-time pad resistance	Achieved	Theorem 4.2
OFV (oracle-free)	Achieved	Theorem 4.3
Geometric cost escalation	Achieved	Lemma 3.10
Layer indistinguishability	Achieved (cond.)	Theorem 4.4
Memory-hard KDF	Empirical	8 MB, ≈2/sec
Confidentiality (IND-CPA)	**Conjectured**	Conjecture 4.5
Formal security reduction	**Open**	Section 7

5. Empirical Results

5.1. Keystream Statistical Tests

Table 4. Keystream statistical tests (4096 B sample, fixed test vectors).

Test	Result	Threshold	Status
Chi-squared uniformity	$\chi^2 = 232$, $p = 0.85$	$p > 0.01$	**Pass**
Pearson autocorr. (lag 1–99)	$\max\lvert r\rvert = 0.041$	< 0.05	**Pass**
Kolmogorov-Smirnov	$p = 0.017$	$p > 0.01$	**Pass**
Byte mean	128.1	127.5 ± 5	**Pass**
Byte std deviation	73.87	73.9 ± 2	**Pass**
NaN/Inf in field	0 occurrences	0	**Pass**
Field alone (no whitener)	$\chi^2 = 1752$, $p \approx 0$	—	**Fails**

5.2. Avalanche Effect

Table 5. Avalanche effect (4096-byte keystream, 4096 bits).

Modification	Bits changed	Percentage
+1 character	2063	50.4%
Case change a→A	2028	49.5%
Trailing space	2013	49.1%
Completely different	2113	51.6%

5.3. KDF Performance

0.53 s per derivation · ≈1.9 attempts/sec · 8 MB RAM/thread. GPU RTX 4090 (24 GB): $\leq$ 3 000 concurrent threads.

5.4. FractalShield Performance

Table 6. FractalShield performance measurements (49-byte message).

Level	Layers	KDF_M sequence	Enc	Dec	Blob	Round-trip
1 Standard	3	$[256, 512, 1024]$	0.35 s	0.11 s	379 B	OK
2 Reinforced	4	$[256, 512, 1024, 2048]$	0.60 s	0.24 s	475 B	OK
3 Maximum	5	$[256, 512, 1024, 2048, 4096]$	1.18 s	0.12 s	571 B	OK

The legitimate user always decrypts at Layer 0 cost (KDF_M = 256) regardless of protection level.

5.5. Layer Statistical Indistinguishability

Table 7. Statistical comparison: real vs. decoy layers.

Property	Real layer	Decoy layers	Distinguishable?
Size	L bytes	L bytes (identical)	No
Chi-squared	$p = 0.85$	$p \approx 0.83$ avg	No
Magic prefix	`MFSU\x04` present	Absent ($p = 1 - N/2^{40}$)	Only with key
Byte entropy	≈ 7.99 bits	≈ 7.98 bits	No

6. Comparison with Existing Systems

Table 8. MFSU-Crypt + FractalShield vs. existing systems.

Property	AES-256-GCM	ChaCha20	Argon2+AES	Honey Enc. [4]	MFSU + FractalShield
Formal security proof	Yes	Yes	Partial	Yes (DTE)	Conj.
Memory-hard KDF	Ext.	Ext.	Yes, 64 MB	No	Yes, 8 MB
No verif. oracle	No	No	No	Partial	**Yes**
Geometric cost	No	No	No	No	**Yes**
Single equation	No	No	No	No	**Yes**
Fractional diffusion	No	No	No	No	**Yes**
Public audit	25+ yr	15+ yr	10+ yr	10+ yr	None yet
Serverless defense	—	—	—	Yes	**Yes**

7. Open Problems and Roadmap

7.1. Open Problems

1. **PRG security:** prove MFSU-keystream is a PRG under a standard computational assumption.
2. **Memory-hardness:** construct DAG $G = (V, E)$ for the scratchpad access pattern; prove $\Omega(M \cdot N)$ sequential space complexity under the parallel ROM [3].
3. **IND-CCA2 of FractalShield:** formalize Theorem 4.3 under a concrete security game with a reduction.
4. **Collision resistance of MFSU-Hash:** bound $\Pr[\text{MFSU-Hash}(x) = \text{MFSU-Hash}(y)]$ for $x \neq y$.
5. **Full NIST STS battery:** apply all 15 tests in [7] to $\geq 10^8$ keystream bits.
6. **Timing side-channel:** formally verify that numpy's FFT is data-independent at the machine-code level, or replace with a constant-time C implementation.

7.2. Certification Roadmap

Year 1 arXiv publication · NIST STS battery · formalize OFV game · C implementation.

Year 2 IND-CPA proof under ROM · memory-hardness DAG proof · independent audit.

Year 3 Public competition submission · IND-CCA2 proof or concrete security bound · mobile optimization (Android/iOS).

8. Conclusion

We presented MFSU-Crypt + FractalShield, a complete symmetric cryptographic system with an adaptive layered defense mechanism, both derived from the MFSU equation (7).

The system provides empirically validated properties (Tables 4–7), four formal security theorems (Theorems 4.1–4.4), and one explicitly marked conjecture (Conjecture 4.5).

FractalShield's principal contribution—oracle-free verification with geometric cost escalation, offline and serverless— addresses a gap in the literature that no existing documented construction fills completely. The mechanism's coherence with the underlying MFSU model (fractal self-complexity under perturbation, Eq. (7)) gives it theoretical elegance beyond its practical utility.

The full implementation is released under the MIT License at `https://github.com/Fracta-Axis/Fractalyx`. We invite public cryptanalysis.

References

[1] NIST (2001). *Advanced Encryption Standard (AES)*. FIPS PUB 197.

[2] Bernstein, D.J. (2008). ChaCha, a variant of Salsa20. *Workshop Record of SASC 2008*.

[3] Alwen, J. & Serbinenko, V. (2015). High Parallel Complexity Graphs and Memory-Hard Functions. *STOC 2015*, pp. 595–603.

[4] Juels, A. & Ristenpart, T. (2014). Honey Encryption: Security Beyond the Brute-Force Bound. *EUROCRYPT 2014*, LNCS 8441, pp. 293–310.

[5] Franco León, M.A. (2025). The Universal Fractal Law: Final Cosmological Validation of the MFSU Model. Preprint.

[6] Planck Collaboration (2018). Planck 2018 results. *Astronomy & Astrophysics*, 641. `https://pla.esac.esa.int`

[7] NIST (2010). A Statistical Test Suite for Random and Pseudorandom Number Generators. *SP 800-22 Rev. 1a*.

[8] Bellare, M. & Rogaway, P. (1993). Random Oracles are Practical. *CCS 1993*, pp. 62–73.

[9] Biryukov, A. & Khovratovich, D. (2016). Argon2: New Generation of Memory-Hard Functions. *IEEE EuroS&P 2016*, pp. 292–302.

[10] Percival, C. (2009). Stronger Key Derivation via Sequential Memory-Hard Functions. *BSDCan 2009*.

[11] Samko, S., Kilbas, A. & Marichev, O. (1993). *Fractional Integrals and Derivatives: Theory and Applications*. Gordon & Breach.

[12] Decreusefond, L. & Ustünel, A.S. (1999). Stochastic Analysis of the Fractional Brownian Motion. *Potential Analysis*, 10(2), 177–214.

A. Complete Parameter Table

Table 9. All system parameters.

Symbol	Value	Derivation	Role
δ_F	0.921	Eq. (1)	Universal fractal deviation
β	1.079	$2 - \delta_F$	Fractional Laplacian order
d_f	2.921	$2 + \delta_F$	Projected fractal dimension
γ	0.921	δ_F	Nonlinear coefficient
σ	0.1	Fixed	Noise intensity
H	0.541	CMB fit	Hurst exponent of fGn
N_{KDF}	2048	Memory target	KDF field size
M_{KDF}	256	Time target	KDF base evolution steps
$N_{\text{KDF}} \cdot M_{\text{KDF}} \cdot 16\text{B}$	8 MB		Scratchpad RAM
N_{ks}	512	Throughput	Keystream field size
n_{steps}	$48+h[0] \bmod 64$		Variable SPDE steps
|`MFSU\x04`|	5 B	$1/2^{40}$ collision	Internal magic prefix
IV, SALT	128 b	Random per enc.	Uniqueness guarantees
MAC	HMAC-SHA3-256	Enc-then-MAC	Integrity & authenticity

B. Test Vectors

Listing 1. Test vector specification.

```
# Base keystream
                              # 16 bytes ASCII
                              # 16 bytes ASCII


# Expected statistics (4096-byte sample)

                           # Pearson r, lags 1-99
                           # ideal: 127.5
                           # percent bits changed (+1 char)

# FractalShield (password='clave_fractal_test', msg=49 bytes)
# Level 1: enc=0.35s  dec=0.11s  size=379B  roundtrip=True
# Level 2: enc=0.60s  dec=0.24s  size=475B  roundtrip=True
# Level 3: enc=1.18s  dec=0.12s  size=571B  roundtrip=True

# Security checks
```

Full executable test suite: `https://github.com/Fracta-Axis/Fractalyx`

C. FractalShield Security Game (Formal)

FractalShield OFV Security Game (Experiment $\mathbf{Exp}_{\mathcal{A}}^{\mathrm{OFV}}$)

Setup: Challenger samples key $k \leftarrow_{\$} \{0,1\}^{\kappa}$, plaintext M, protection level ℓ. Computes $C \leftarrow \textsc{FractalShield}.\mathrm{Enc}(M, k, \ell)$. Sends C to adversary $\mathcal{A}$.

Query phase: $\mathcal{A}$ submits candidate keys $k'_1, k'_2, \ldots$ For each k'_i:

1. $r_i \leftarrow \textsc{FractalShield}.\mathrm{Dec}(C, k'_i)$ [Full KDF + layer decryption + magic check]
2. If $r_i \neq \perp$: challenger returns Found.

Cost of each query: $C_{\mathrm{attacker}}(\ell)$ (Lemma 3.10).

Success: $\mathcal{A}$ outputs correct k, or finds any k' such that $\textsc{FractalShield}.\mathrm{Dec}(C, k') \neq \perp$.

Advantage: $\mathbf{Adv}_{\mathcal{A}}^{\mathrm{OFV}}(\kappa) = \Pr[\mathbf{Exp}_{\mathcal{A}}^{\mathrm{OFV}} = 1]$.

By Theorem 4.3 and Lemma 3.9, this advantage is bounded by $\varepsilon_{\mathrm{SHA3}} + \varepsilon_{\mathrm{PRG}} + N/2^{40}$, all of which are negligible under standard assumptions.